ΗΥ-225 Άσκηση 4

Σαράντης
Σαρανινός
CSD 4511

4α

Σε περίπτωση που "ξοδέψουμε" και τους 8 διαθέσιμους συνδυασμούς του πεδίου op για εντολές I-Format τότε δεν θα υπάρχει διαθέσιμος συνδυασμός για εντολή R-Format δεδομένου ότι το πεδίο op έχει 3 bits. Αν στην ίδια περίπτωση προσπαθήσουμε να υιοθετήσουμε μια R-Format εντολή θα παρερμηνευθεί ως I-Format και τα πεδία R και funct μετά τον opcode θα ληφθούν ως Immediate, καθώς όλοι οι συνδυασμοί op εκφράζουν I-Format εντολές. Για την εντολή R-Format "RR" με op=001 και funct = 10 στον καταχωρητή x6, δηλαδή "RR x6", αρχικά θα εκφράσουμε τον καταχωρητή στα 3 bits ως 110, οπότε η "RR x6" θα αναπαρασταθεί ως

10	110	001
funct	x6	op

~~001 110 001~~
~~funct x6 op~~

και θα παρερμηνευθεί με την ίδια και απαράλλακτη I-Format εντολή

10110	001
Imm	op

με op=001 και Imm = 10110 που εκφράζει τον αριθμό 22. Το ίδιο ισχύει και για οποιονδήποτε συνδυασμό R-Format όπου τα συνολικά 5 bits των R και funct θα μετατραπούν στην αντίστοιχη ~~αριθμο~~ σταθερά Imm από 0 έως 31.

4β) Αν δεσμεύουμε 7 από 8 συνδυασμούς op για I-Format θα μπορούμε να έχουμε 4 R-Format εντολές.

I-Format	
op	εντολή
000	ni
001	pi
010	qi
011	ri
100	si
101	ti
110	vi

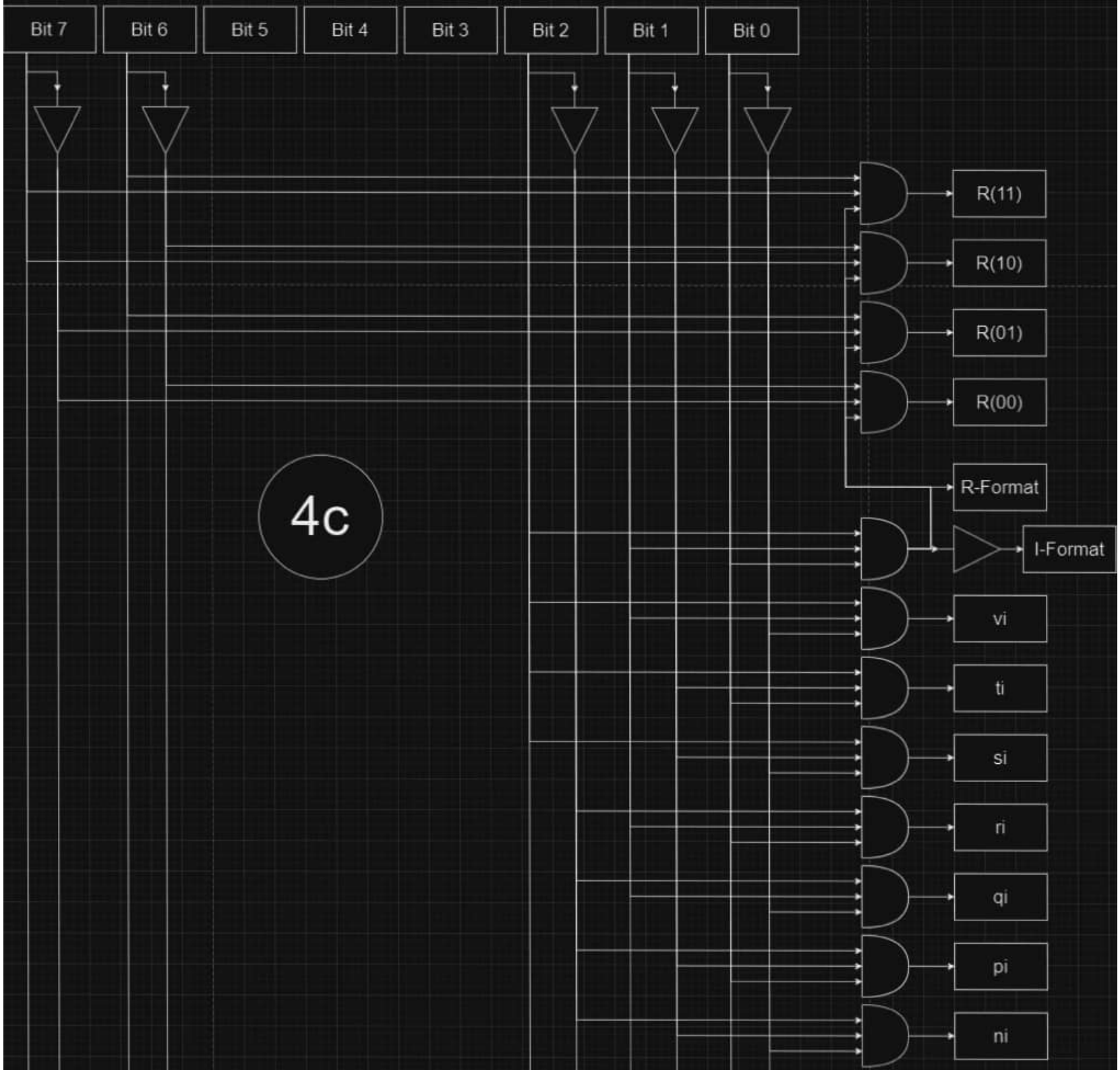
R-Format	
op	Funct
111	00
111	01
111	10
111	11

Δεν μπορούμε να έχουμε περισσότερες από 4 εντολές R καθώς δεν υπάρχει άλλος συνδυασμός για 3 bits op και 2 bits Funct.

4δ) Αν έχουμε 6 εντολές I-Format, μπορούμε να έχουμε έως και 8 εντολές R-Format

op	
000	I-Format
001	I-Format
010	I-Format
011	I-Format
100	I-Format
101	I-Format
110	R-Format →
111	R-Format →

op	Funct
110	00
110	01
110	10
110	11
111	00
111	01
111	10
111	11


Bit 7
Bit 6
Bit 5
Bit 4
Bit 3
Bit 2
Bit 1
Bit 0
R(11)
R(10)
R(01)
R(00)
R-Format
I-Format
vi
ti
si
ri
qi
pi
ni
4c

ΗΥ-225 Σειρά 5
Σαραπινός Σαράντης csd4511

Άσκηση 2η

α) Ο RISC-V δεν έχει εντολές ble και bgt διότι αυτές οι πράξεις καλύπτονται από τις ήδη υπάρχουσες εντολές.

Έστω i στον x22 και j στον x23.
Για να εκφράσουμε $i \leq j$ θα χρησιμοποιήσουμε
bge x23, x22, label
και για να εκφράσουμε $i > j$
blt x23, x22, label.

Ουσιαστικά, χρησιμοποιούμε τις ήδη υπάρχουσες εντολές με αντιστραμμένη σειρά ~~φορά~~ των καταχωρητών.

β) Δεν υπάρχουν εντολές bequ και bneu για να συγκρίνουν απροσήμαστους διότι δεν χρειάζονται, καθώς οι beq και bne ~~κάνουν~~ καλύπτουν αυτήν την ανάγκη συγκρίνοντάς τους ως προσημασμένους. Δεν υπάρχει κανένα πρόβλημα με αυτή τη σύγκριση διότι αν δύο αριθμοί είναι ίσοι ως προσημασμένοι θα είναι ίσοι και ως απροσήμαστοι. Κάτι τέτοιο βέβαια δεν είναι καν υλοποιήσιμο διότι δεν περισσεύουν αρκετά bits ώστε να χωρέσει ένας Immidiate αριθμός σε εντολή branch, οι εντολές B-Format των 32 bits χρησιμοποιούν τα 5 για τον πρώτο καταχωρητή, για τον opcode 7, για το Funct 3 και 12 για το label οπότε δεν

χωράει ένας Immidiate στα 5 bits που περισσεύουν.

γ) Δεν υπάρχουν εντολές που να συγκρίνουν το περιεχόμενο ενός καταχωρητή με μια σταθερά διότι αυτό μπορεί να συμβεί θέτοντας την σταθερά σε έναν δεύτερο καταχωρητή και έπειτα συγκρίνοντας μεταξύ τους τους καταχωρητές.

π.χ. addi x23, x0, 18
blt x22, x23, label

δ) Για να εκφράσουμε το "branch if(i ≤ 13)" αρχικά θα θέσουμε έναν καταχωρητή για τον i και έπειτα θα βάλουμε την τιμή 13 σε έναν δεύτερο, θα τους συγκρίνουμε χρησιμοποιώντας την εντολή bge με αποτελεσμένη σειρά των καταχωρητών και αν ισχύει η σύγκριση θα μεταφερθούμε στο label.

# i → x22, temp → x23
addi x23, x0, 13 # temp = 13
bge x23, x22, label # if (13 ≥ i) goto label

Scanned with CamScanner

Άσκηση 5η

```
        add  t1, x0, x0
loop54: slli t0, t1, 3
        add  t0, s0, t0
        ld   t0, 0(t0)
        addi t1, t1, 1
        bne  t0, t2, loop54   #if(table[i] ≠ key) goto Loop54
cont54: . . .
```

Αν η επανάληψη συμβαίνει 10 φορές τότε στον παλαιό κώδικα εκτελούνται 58 εντολές ενώ στον καινούριο 50 εντολές.

Άσκηση 11η

```
#i) B = (i < j)
slt  x24, x21, x22
#ii) B = (i ≥ j)
slt  x24, x21, x22
xori x24, x24, 1
#iii) B = (i > j)
slt  x24, x22, x21
#iv) B = (i ≤ j)
slt  x24, x22, x21
xori x24, x24, 1
#v) B = (i < const)
slt  x24, x21, CONST
#vi) B = (i ≥ CONST)
slt  x24, x21, CONST
xori x24, x24, 1
```

Scanned with CamScanner

```
# vii) B = (i > CONST)
slti x24, x21, CONST+1
xori x24, x24, 1
# viii) B = (i ≤ CONST)
slti x24, x21, CONST+1

# a) B = (i == j)
sub x24, x21, x22
sltiu x24, x24, 1

# b) B = (i != j)
sub x24, x21, x22
sltiu x24, x24, 1
xori x24, x24, 1

# c) B = (i == const)
addi x24, x21, -CONST
sltiu x24, x24, 1

# d B = (i != CONST)
addi x24, x21, -CONST
sltiu x24, x24, 1
xori x24, x24, 1
```

Scanned with CamScanner